# Generative AI for Synthetic Financial Data

Instructor: Chao ZHANG

March 17, 2026

Reinforcement Learning Recap

Introduction to Generative AI
- Autoencoders
- Generative Adversarial Networks (GAN)

Applications in Finance

Challenges of Generative AI in Finance

# Reinforcement Learning Algos

- Value-based methods
  - focus on estimating *a value function (usually the Q-function)* and then deriving a policy from it (typically by choosing the action with the highest estimated value).
  - Q-Learning (Off-policy)
  - SARSA (On-policy)
- Policy-based methods
  - focus directly on *learning the policy (i.e., the mapping from states to actions)* without explicitly maintaining a value function.
  - REINFORCE
- Hybrid Approaches: Actor-Critic
- Examples: Super Mario Bros, Mario Cart

# RL in Games vs Finance

**Question:**

Why has Reinforcement Learning achieved remarkable success in games, but much more limited success in financial markets?

# RL in Games vs Finance

**Question:**

Why has Reinforcement Learning achieved remarkable success in games, but much more limited success in financial markets?

Examples of RL success:

- AlphaGo / AlphaZero (Go, Chess, Shogi)
- Deep RL agents trained via massive self-play

# RL in Games vs Finance

**Question:**

Why has Reinforcement Learning achieved remarkable success in games, but much more limited success in financial markets?

Examples of RL success:

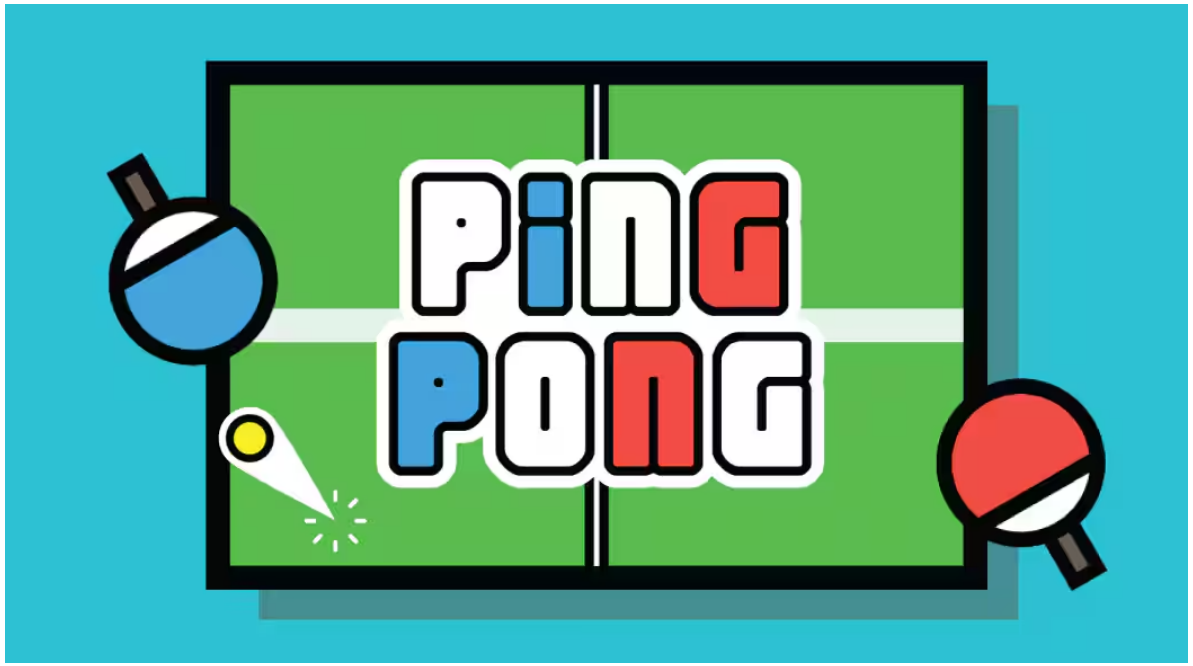
- AlphaGo / AlphaZero (Go, Chess, Shogi)
- Deep RL agents trained via massive self-play

However, applying RL to financial markets remains challenging.

**Key differences between games and financial markets**

# 1. Non-Stationarity and Multi-Agent Dynamics

- **Games:**
  - Well-defined, consistent rules with stationary dynamics.
  - Opponents' strategies are often fixed or bounded within the game engine.
- **Finance:**
  - Highly non-stationary; market distributions shift unpredictably due to macro events.
  - An inherently multi-agent environment where adaptive participants with hidden strategies create continuous adversarial dynamics.



# 2. Complex Objectives and Market Frictions

- **Games:**
  - Clear, immediate, and noise-free reward signals (e.g., points, win/loss).
- **Finance:**
  - Multi-dimensional objectives: balancing expected returns with rigorous risk management (e.g., drawdown, volatility).
  - Severe market frictions: performance evaluation and RL reward design must account for transaction costs, slippage, and market impact, making the reward signal extremely noisy and delayed.

# 3. The Simulation Bottleneck

- **Games:**
  - High-fidelity simulators allow agents to play millions of games, generating infinite, cost-free data for RL convergence.
- **Finance:**
  - We lack a "perfect market simulator." Live trading data is limited, historical data represents only a single realized path, and trial-and-error in real markets involves unacceptable financial ruin.

*The critical gap leading to Generative AI:*

# From RL to Market Simulators via Generative AI

Among these challenges, a key bottleneck is:

**The lack of reliable financial market simulators.**

# From RL to Market Simulators via Generative AI

Among these challenges, a key bottleneck is:

**The lack of reliable financial market simulators.**
If we could build realistic market simulators, RL agents could be trained through simulated interactions.

# From RL to Market Simulators via Generative AI

Among these challenges, a key bottleneck is:

**The lack of reliable financial market simulators.**
If we could build realistic market simulators, RL agents could be trained through simulated interactions.

**Idea:**
Learn the market dynamics directly from historical data.

$$p(x_{1:T})$$

or

$$p(x_{t+1} \mid x_{1:t})$$

# From RL to Market Simulators via Generative AI

Among these challenges, a key bottleneck is:

**The lack of reliable financial market simulators.**
If we could build realistic market simulators, RL agents could be trained through simulated interactions.

**Idea:**
Learn the market dynamics directly from historical data.

$$p(x_{1:T})$$

or

$$p(x_{t+1} \mid x_{1:t})$$

Generative AI provides a powerful framework to learn such data-generating processes.

Reinforcement Learning Recap

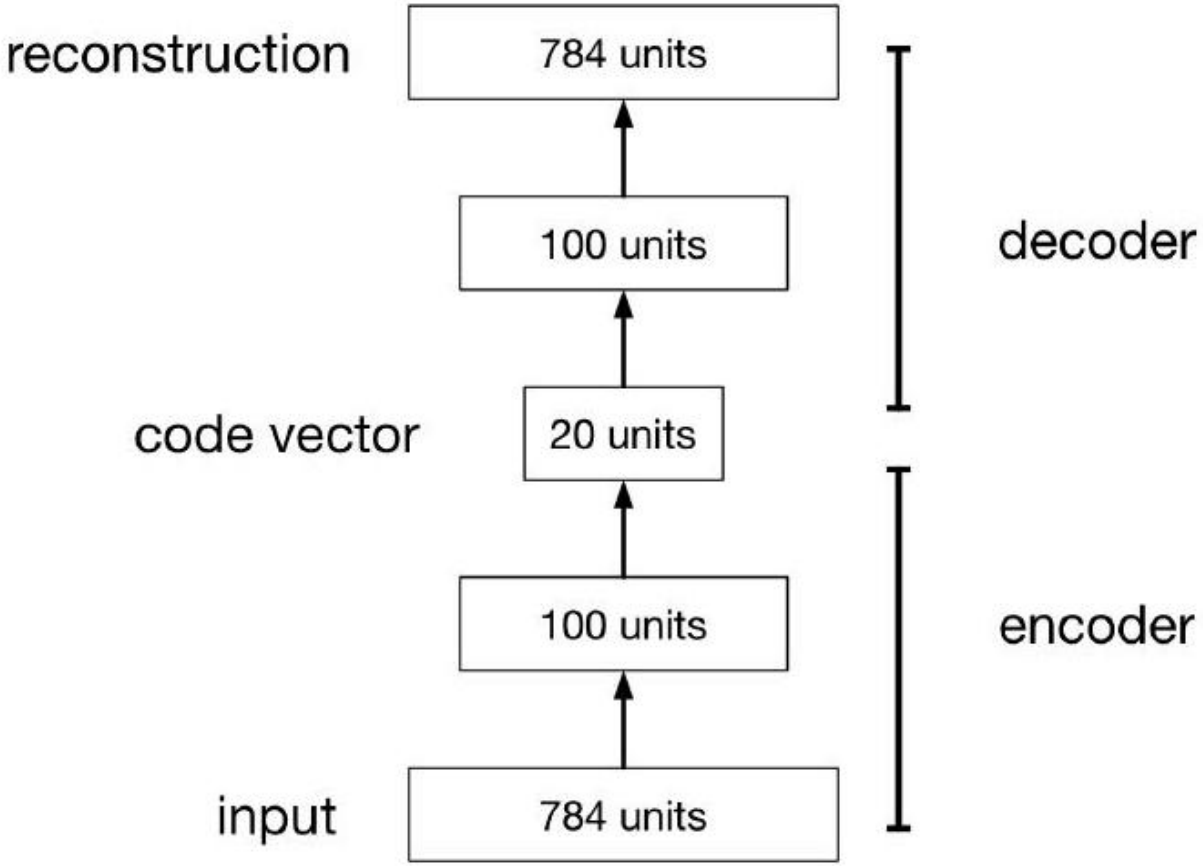
Introduction to Generative AI
- Autoencoders
- Generative Adversarial Networks (GAN)

Applications in Finance

Challenges of Generative AI in Finance

# Autoencoders

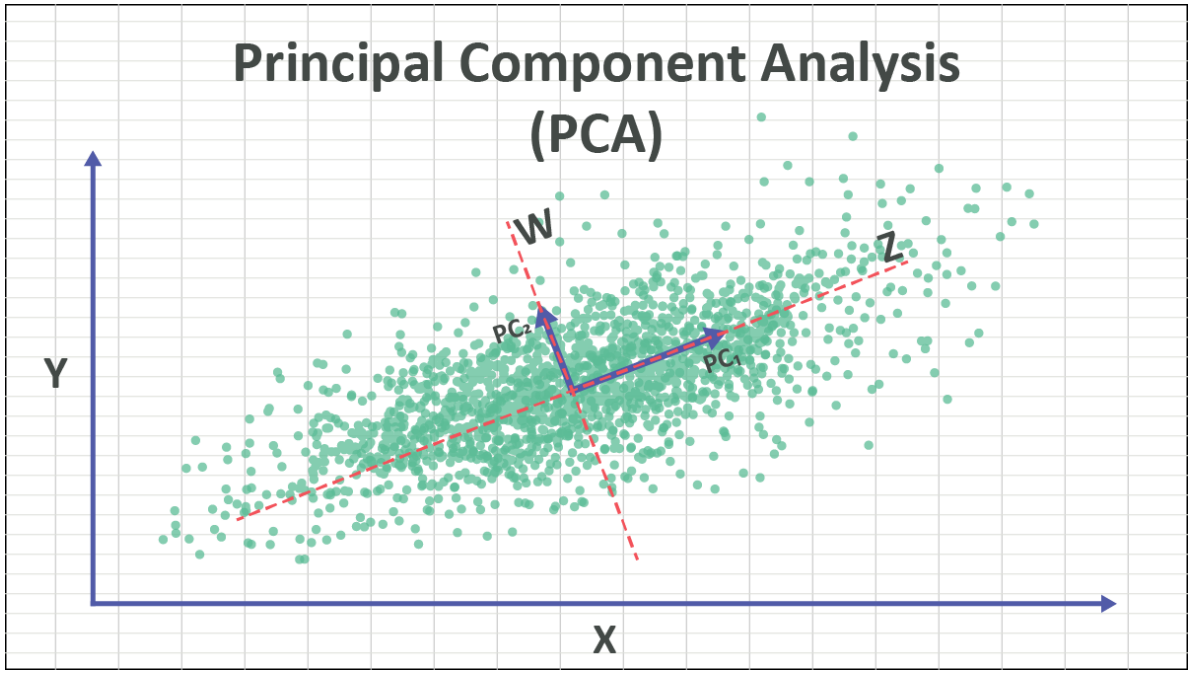
- An autoencoder is a feed-forward neural net whose job it is to take an input **x** and predict **x**.
- To make this non-trivial, we need to add a bottleneck layer whose dimension is much smaller than the input.



# Why autoencoders?

- Map high-dimensional data to two dimensions for visualization
- Compression (i.e. reducing the file size)
- Learn abstract features in an unsupervised way so you can apply them to a supervised task
- Unlabled data can be much more plentiful than labeled data
- Learn a semantically meaningful representation where you can, e.g., interpolate between different images.

# Principal Component Analysis




PCA seeks to find the principal components (PCs) that capture the maximum variance in the data. Mathematically:

$$\min_{W} \sum_{i} \left\| x_i - WW^T x_i \right\|^2$$

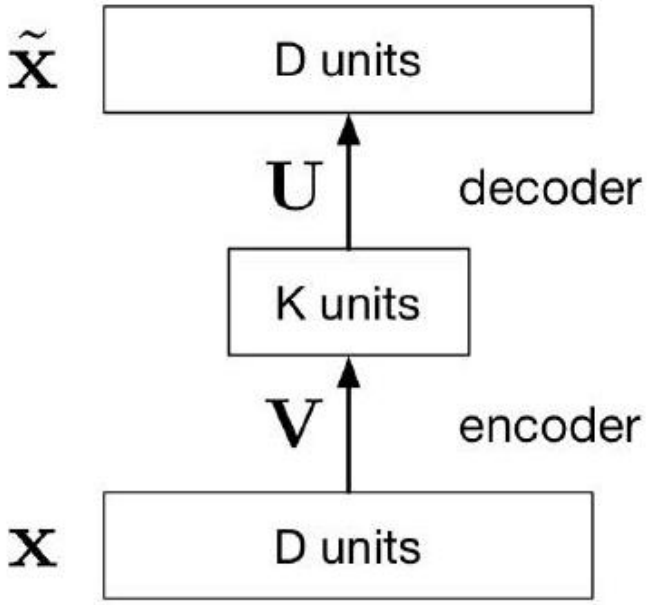
subject to $W^T W = I$, ensuring that the PCs are orthonormal.

# Principal Component Analysis

- The simplest kind of autoencoder has one hidden layer, linear activations, and squared error loss.

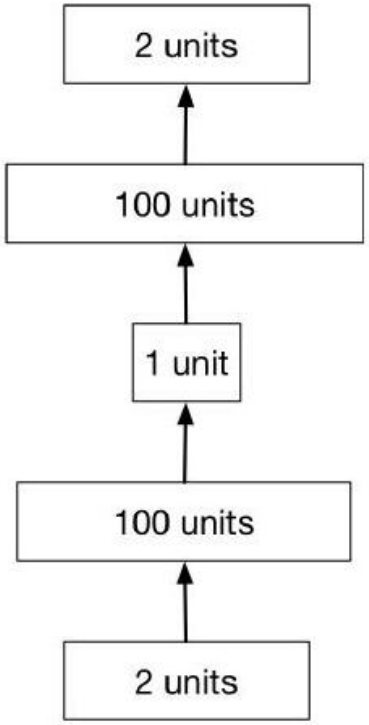
$$\mathcal{L}(\mathbf{x}, \tilde{\mathbf{x}}) = \|\mathbf{x} - \tilde{\mathbf{x}}\|^2$$

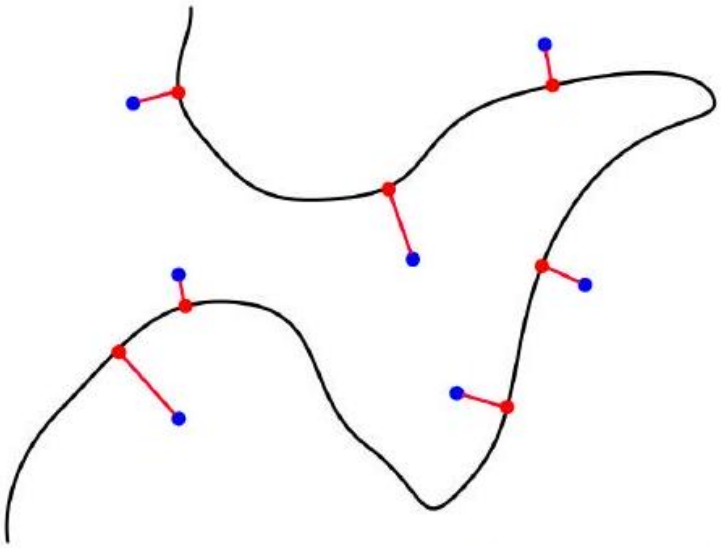
- This network computes $\tilde{\mathbf{x}} = \mathbf{UVx}$, which is a linear function.
- Suppose $K < D$: $V$ maps $\mathbf{x}$ to a K-dimensional space, so it's doing dimensionality reduction.
- The output must lie in a K-dimensional subspace, namely the column space of $\mathbf{U}$.



# Deep Autoencoders

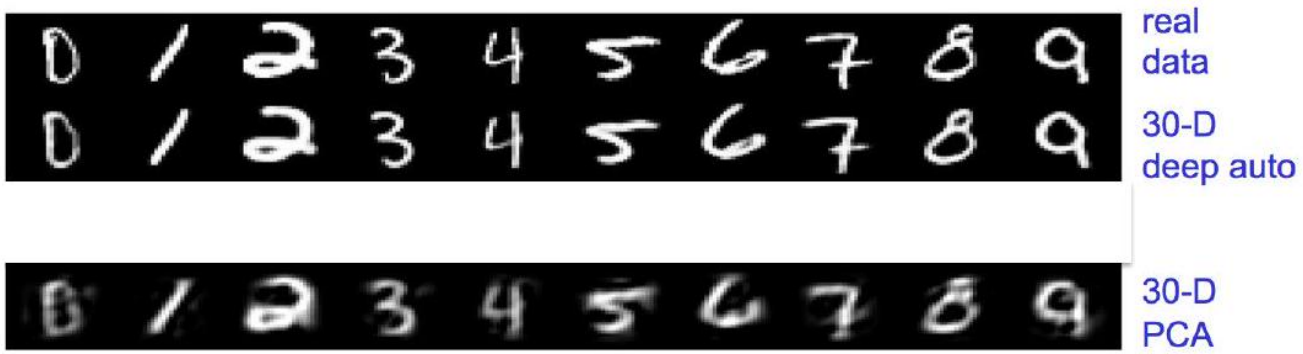
- Deep nonlinear autoencoders learn to project the data, not onto a subspace, but onto a nonlinear manifold
- This manifold is the image of the decoder.
- This is a kind of nonlinear dimensionality reduction.



# Deep Autoencoders

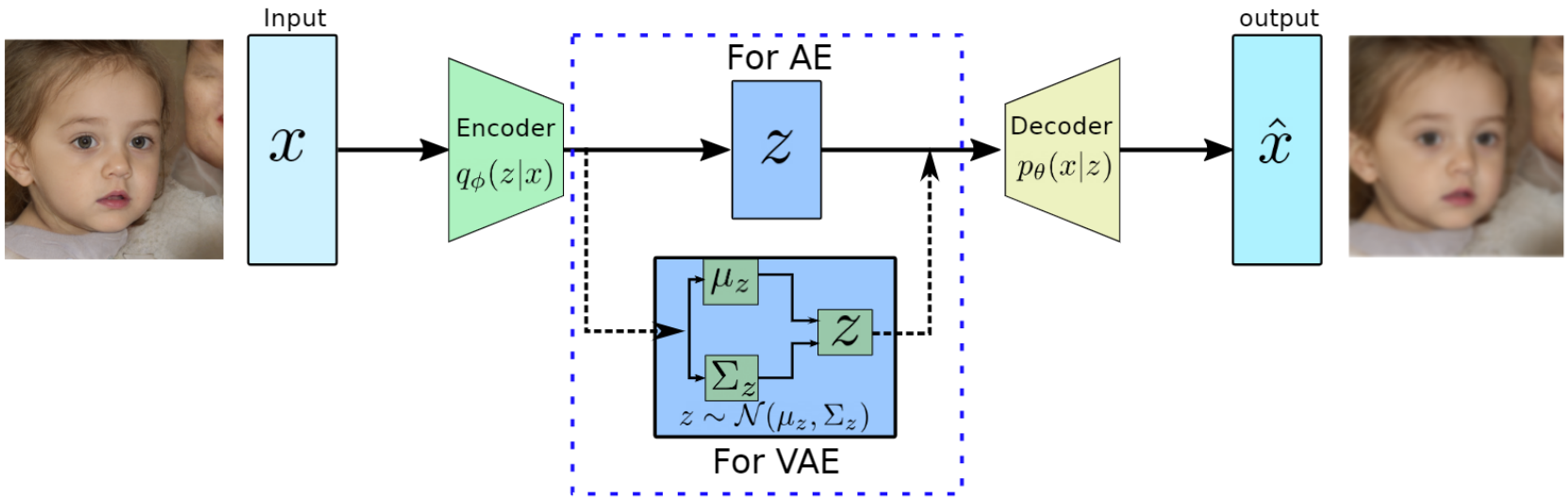
- Nonlinear autoencoders can learn more powerful codes for a given dimensionality, compared with linear autoencoders (PCA)



# Limitations of Deep Autoencoders

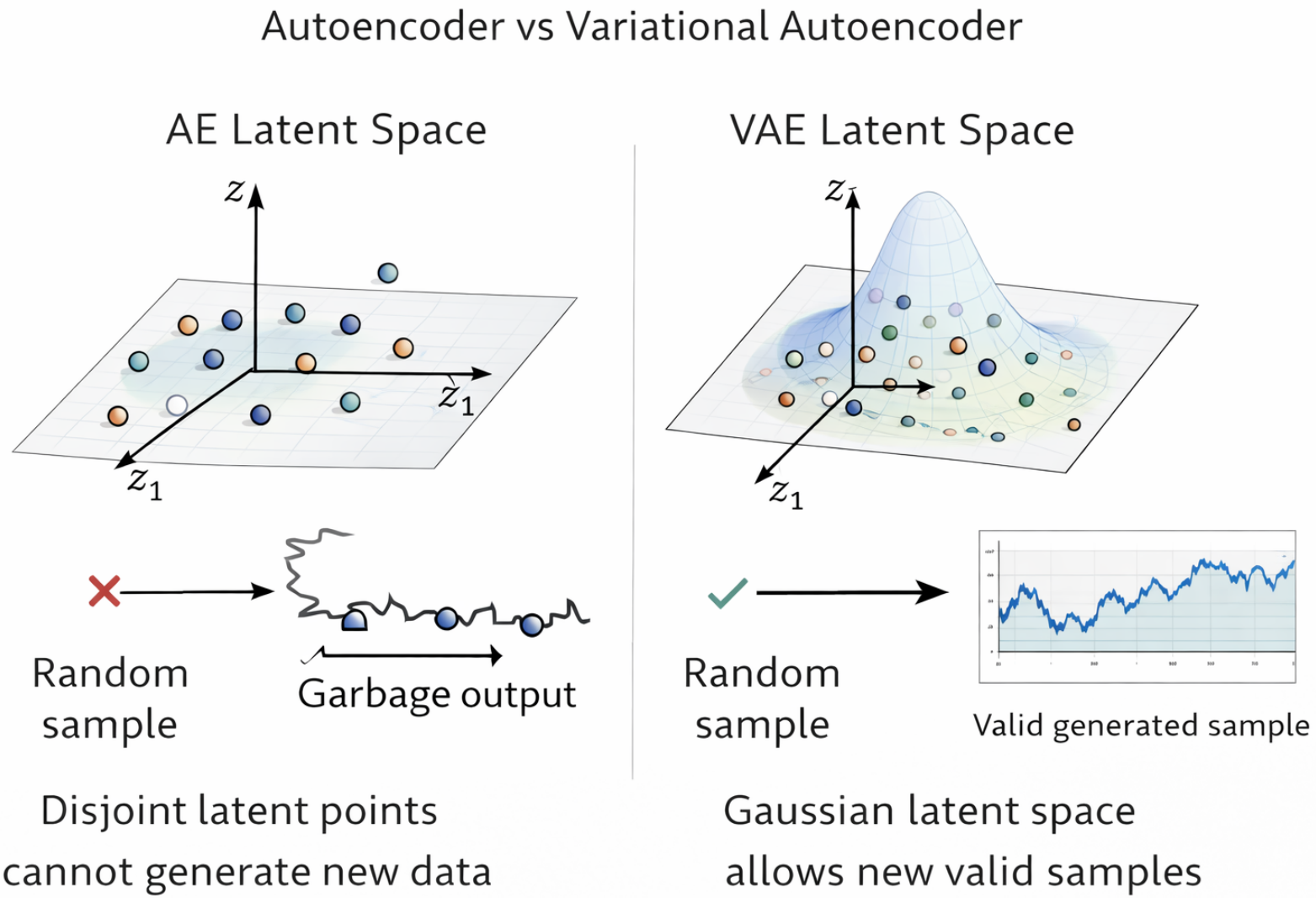
- Focus on minimizing reconstruction loss, but the latent space may not be meaningful or interpretable
  - For example, it may simply copy pixel-level information without understanding object boundaries or textures.
- They're not generative models, so they don't define a distribution
- Memorization Risk: Autoencoders tend to memorize training data rather than learning generalizable features.
- The choice of architecture significantly impacts performance.
  - How to choose the latent dimension?

# VAE




- AE learns a latent representation.
- VAE learns a latent distribution.

# VAE


Autoencoder vs Variational Autoencoder
AE Latent Space
VAE Latent Space
$z$
$z_1$
$z_1$
$z$
$z_1$
Random sample
Garbage output
Random sample
Valid generated sample
Disjoint latent points cannot generate new data
Gaussian latent space allows new valid samples

# Observation Model

A probabilistic model describes how the observed data is generated from latent variables.
Consider training a generator with maximum likelihood.

$$\underbrace{p(\mathbf{x})}_{\text{actual data space}} = \int \underbrace{p(\mathbf{z})}_{\text{latent space}} \underbrace{p(\mathbf{x} \mid \mathbf{z})}_{\text{conditional prob}} \, \mathrm{d}\mathbf{z}$$

**Generator modeled as a neural network**

$$p(\mathbf{x} \mid \mathbf{z}) = \mathcal{N}\left(\mathbf{x}; G_{\boldsymbol{\theta}}(\mathbf{z}), \eta \mathbf{I}\right)$$

where $G_{\boldsymbol{\theta}}$ is the function computed by the decoder with parameters $\boldsymbol{\theta}$.

# VAE target

- VAEs aim to maximize the likelihood of the data $p(x)$.

$$p(x) = \int p(x|z)p(z)\,dz$$

- Direct maximization of $p(x)$ is intractable.
- This intractability arises from integrating over the latent space $z$ and the decoder function $G_\theta(\mathbf{z})$ is very complicated

**Challenges:**

- High Dimensionality
- Non-Analytical Forms
- Computational Cost of numerical integration.

# Variational Inference

- Instead, we will try to maximize a lower bound on $\log p(\mathbf{x})$.
- Jensen's Inequality: for a convex function $h$ of a random variable $X$,

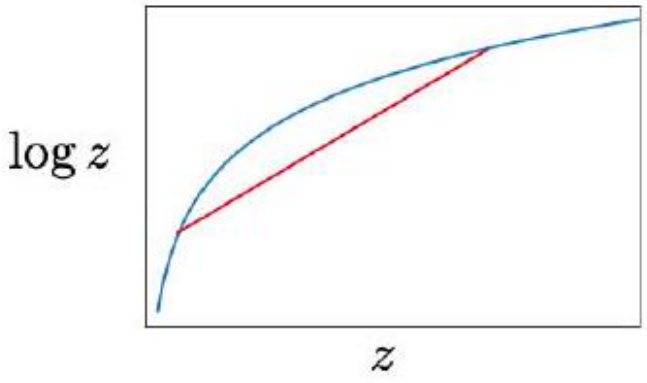
$$\mathbb{E}[h(X)] \geq h(\mathbb{E}[X])$$

- if $h$ is concave (i.e. $-h$ is convex),

$$\mathbb{E}[h(X)] \leq h(\mathbb{E}[X])$$

**Example:** The function $\log z$ is concave/

$$\mathbb{E}[\log X] \leq \log \mathbb{E}[X]$$



# Variational Inference

- Suppose we have some distribution $q(\mathbf{z} \mid \mathbf{x})$. (We'll see later where this comes from.)
- We use Jensen's Inequality to obtain the lower bound.

$$
\begin{aligned}
\log p(\mathbf{x}) &= \log \int p(\mathbf{z}) p(\mathbf{x} \mid \mathbf{z}) \mathrm{d}\mathbf{z} \\
&= \log \int q(\mathbf{z} \mid \mathbf{x}) \frac{p(\mathbf{z})}{q(\mathbf{z} \mid \mathbf{x})} p(\mathbf{x} \mid \mathbf{z}) \mathrm{d}\mathbf{z} \\
&\geq \int q(\mathbf{z} \mid \mathbf{x}) \log \left[ \frac{p(\mathbf{z})}{q(\mathbf{z} \mid \mathbf{x})} p(\mathbf{x} \mid \mathbf{z}) \right] \mathrm{d}\mathbf{z} \qquad \text{(Jensen's Inequality)} \\
&= \mathbb{E}_q \left[ \log \frac{p(\mathbf{z})}{q(\mathbf{z} \mid \mathbf{x})} \right] + \mathbb{E}_q [\log p(\mathbf{x} \mid \mathbf{z})]
\end{aligned}
$$

$\Rightarrow$ Evidence Lower BOund (ELBO)

# Variational Inference

- The ELBO provides a tractable lower bound on $\log p(x)$.
- The first term we'll look at is $\mathbb{E}_q[\log p(\mathbf{x} \mid \mathbf{z})]$
- Since we assumed a Gaussian observation model,

$$
\begin{aligned}
\log p(\mathbf{x} \mid \mathbf{z}) &= \log \mathcal{N}\left(\mathbf{x}; G_\theta(\mathbf{z}), \eta \mathbf{I}\right) \\
&= \log \left[\frac{1}{(2\pi\eta)^{D/2}} \exp\left(-\frac{1}{2\eta}\left\|\mathbf{x} - G_\theta(\mathbf{z})\right\|^2\right)\right] \\
&= -\frac{1}{2\eta}\left\|\mathbf{x} - G_\theta(\mathbf{z})\right\|^2 + \text{ const}
\end{aligned}
$$

- So this term is the expected squared error in reconstructing $\mathbf{x}$ from $\mathbf{z}$. We call it the reconstruction term.
- Through the reconstruction term, the model learns to generate data that closely resembles the original input.

# Variational Inference

- The second term is $\mathbb{E}_q\left[\log \frac{p(z)}{q(\mathbf{z}|\mathbf{x})}\right]$.
- This is just $-\mathrm{D}_{\mathrm{KL}}(q(\mathbf{z} \mid \mathbf{x}) \| p(\mathbf{z}))$, where $\mathrm{D}_{\mathrm{KL}}$ is the Kullback-Leibler (KL) divergence

$$\mathrm{D}_{\mathrm{KL}}(q(\mathbf{z} \mid \mathbf{x}) \| p(\mathbf{z})) \triangleq \mathbb{E}_q\left[\log \frac{q(\mathbf{z} \mid \mathbf{x})}{p(\mathbf{z})}\right]$$

- KL divergence is a widely used measure of distance between probability distributions, though it doesn't satisfy the axioms to be a distance metric.
- Typically, $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$. Hence, the KL term encourages $q$ to be close to $\mathcal{N}(\mathbf{0}, \mathbf{I})$.
- regularizing the latent space and encouraging it to capture meaningful, generalizable features.

# Variational Inference

- Hence, we're trying to maximize the variational lower bound, or variational free energy:

$$\log p(\mathbf{x}) \geq \underbrace{\mathbb{E}_{q(z|x)}[\log p(x \mid z)]}_{\text{reconstruction term}} - \underbrace{KL(q(z \mid x) \| p(z))}_{\text{regularization}}$$

# Variational Inference

- Hence, we're trying to maximize the variational lower bound, or variational free energy:

$$\log p(\mathbf{x}) \geq \underbrace{\mathbb{E}_{q(z|x)}[\log p(x \mid z)]}_{\text{reconstruction term}} - \underbrace{KL(q(z \mid x) \| p(z))}_{\text{regularization}}$$

- without KL $\approx$ Autoencoder
- KL Regularization makes the latent space continuous and smooth.
- without Reconstruction $\Rightarrow$ posterior collapse
- Reconstruction makes the latent variable informative.

- **We'd like to choose $q$ to make the bound as tight as possible.**

# Tractable Family of Distributions

**Common Choice: Gaussian Distribution**

- Assume $q(z|x) = \mathcal{N}\big(z; \mu(x), \mathrm{diag}(\sigma^2(x))\big)$.
- Provides a closed-form KL divergence with the prior.
- Benefits:
  - Easy sampling.
  - Tractable computations.

**Expressiveness:**

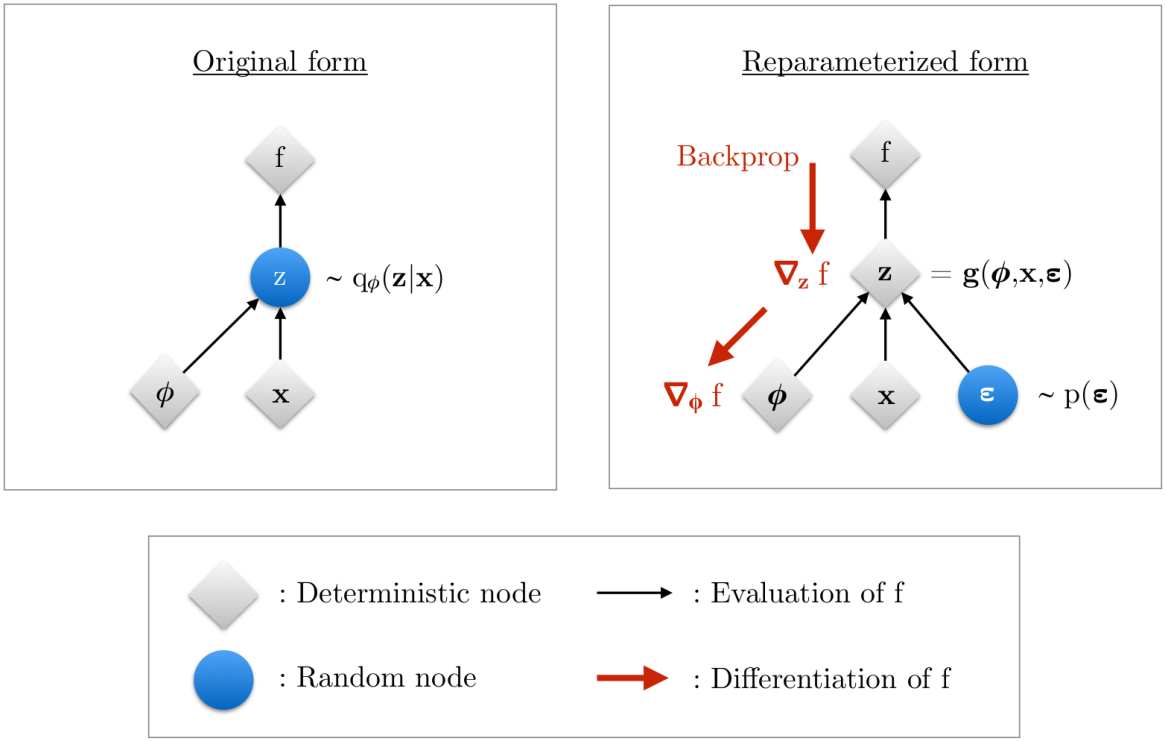
- A simple Gaussian might not capture all complexities of $p(z|x)$.
- Alternatives include mixtures of Gaussians or using normalizing flows.

# Reparameterization Trick

- $q(z \mid x)$ is parameterized by a neural network (encoder) in VAE.
- Direct sampling from $q_\phi(z|x)$ prevents gradients from propagating through $z$.
- Use the reparameterization trick to make it differentiable:

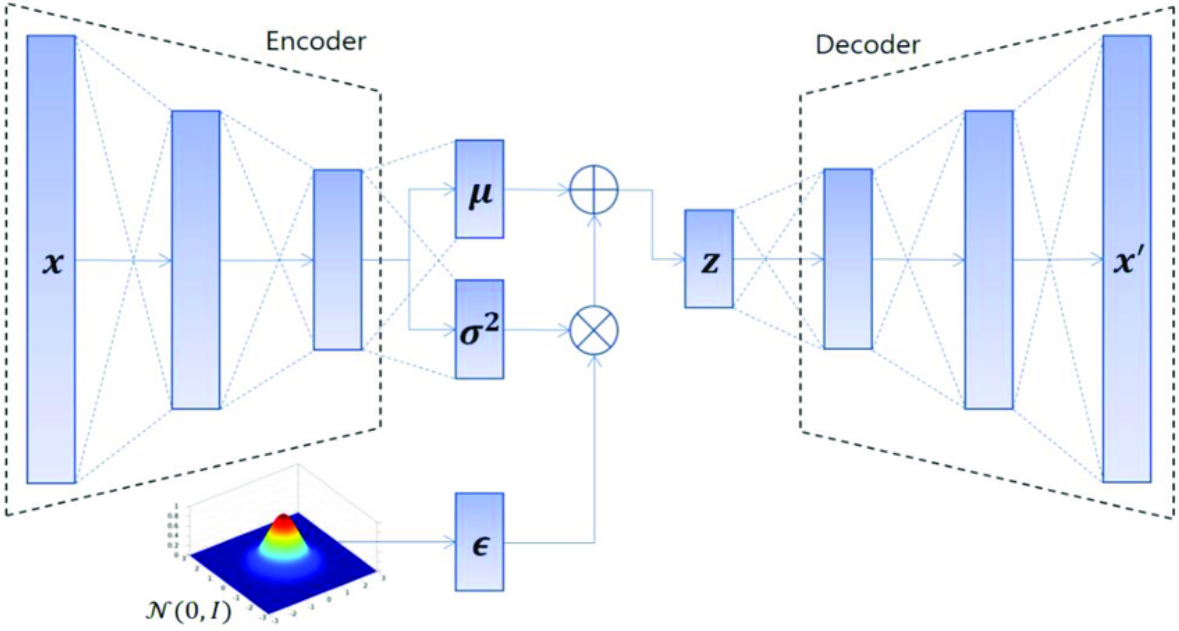
$$z = \mu(x) + \sigma(x) \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

- Enables backpropagation and efficient learning.



# VAE Structure

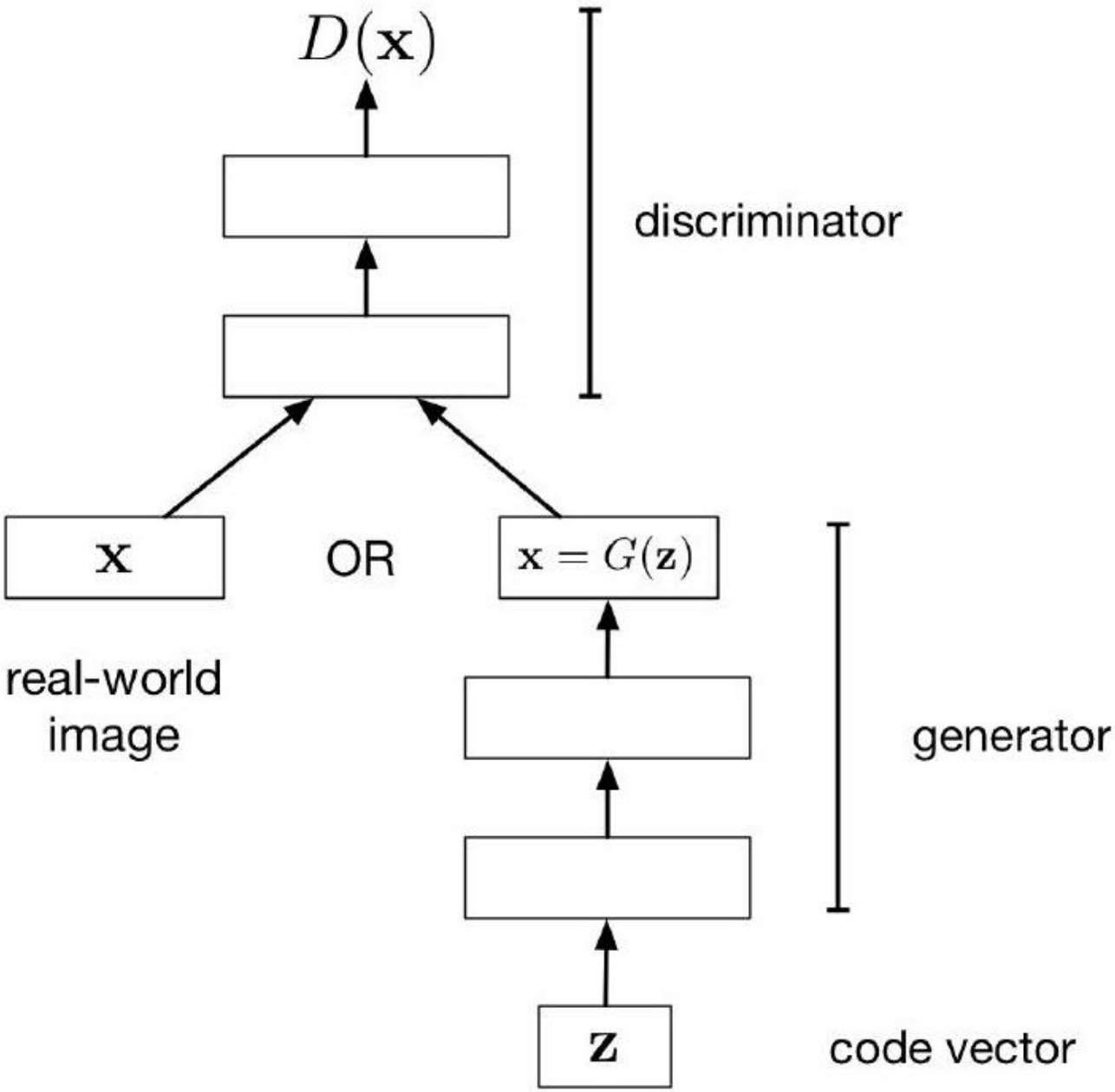
- **Encoder:** Maps input data $x$ to a latent distribution $q(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}(\mu, \sigma^2)$.
- **Latent Space:** Regularized to follow a standard normal distribution.
- **Decoder:** Generates data from sampled latent variables $z$ according to $p(\mathbf{x} \mid \mathbf{z})$.



# Generative Adversarial Networks

- The idea behind Generative Adversarial Networks (GANs): train two different networks
- The generator network tries to produce realistic-looking samples
- The discriminator network tries to figure out whether an image came from the training set or the generator network
- The generator network tries to fool the discriminator network

# GAN


$D(\mathbf{x})$
discriminator
$\mathbf{x}$
OR
$\mathbf{x} = G(\mathbf{z})$
real-world image
generator
$\mathbf{z}$
code vector

# Generative Adversarial Networks

- Let $D$ denote the discriminator's predicted probability of being data
- Discriminator's cost function: cross-entropy loss for task of classifying real vs. fake images

$$\mathcal{J}_D = \mathbb{E}_{\mathbf{x}\sim\mathcal{D}}[-\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}}[-\log(1 - D(G(\mathbf{z})))]$$

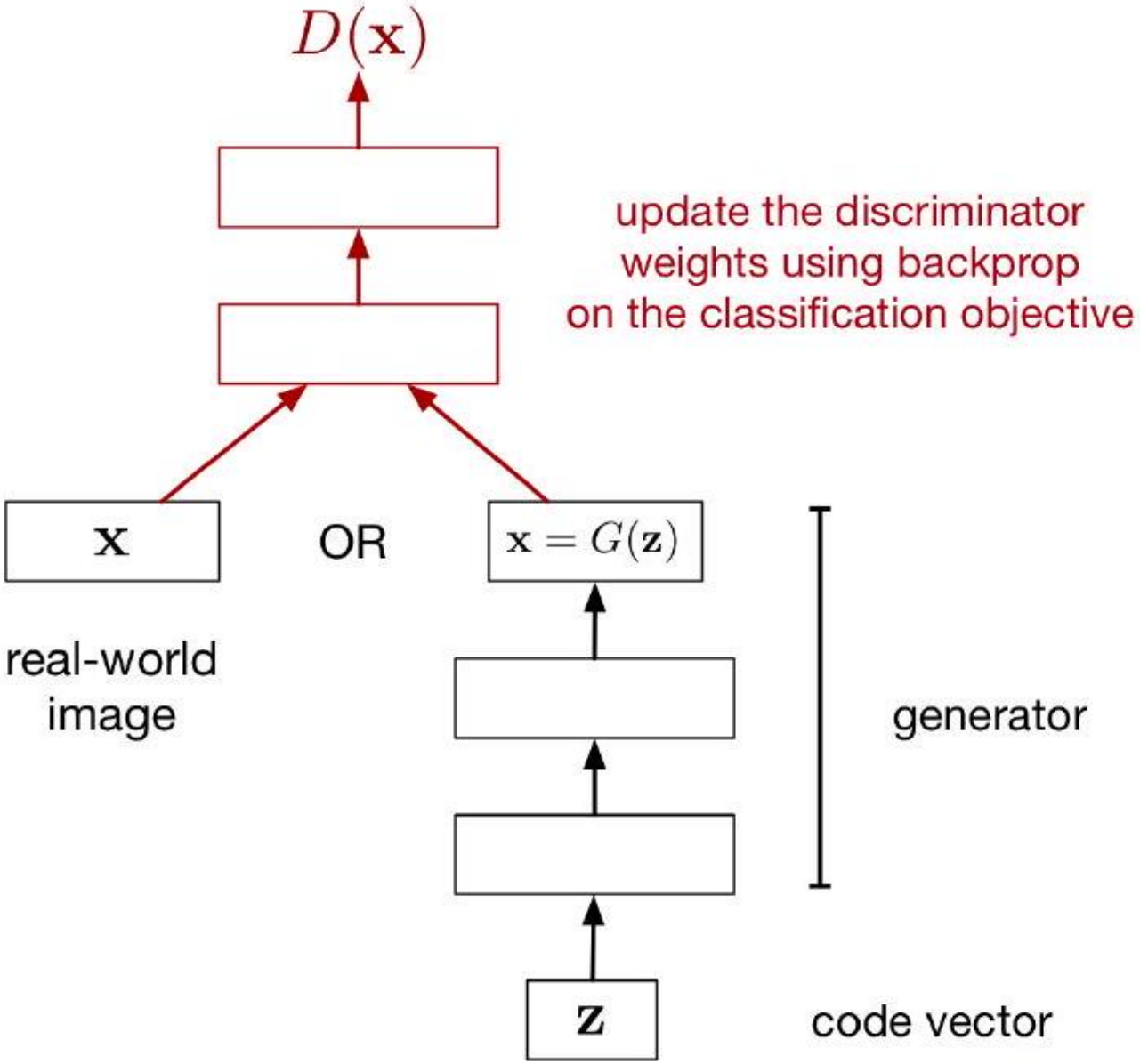
- One possible cost function for the generator: the opposite of the discriminator's

$$\begin{aligned}\mathcal{J}_G &= -\mathcal{J}_D \\ &= \text{ const } + \mathbb{E}_{\mathbf{z}}[\log(1 - D(G(\mathbf{z})))]\end{aligned}$$

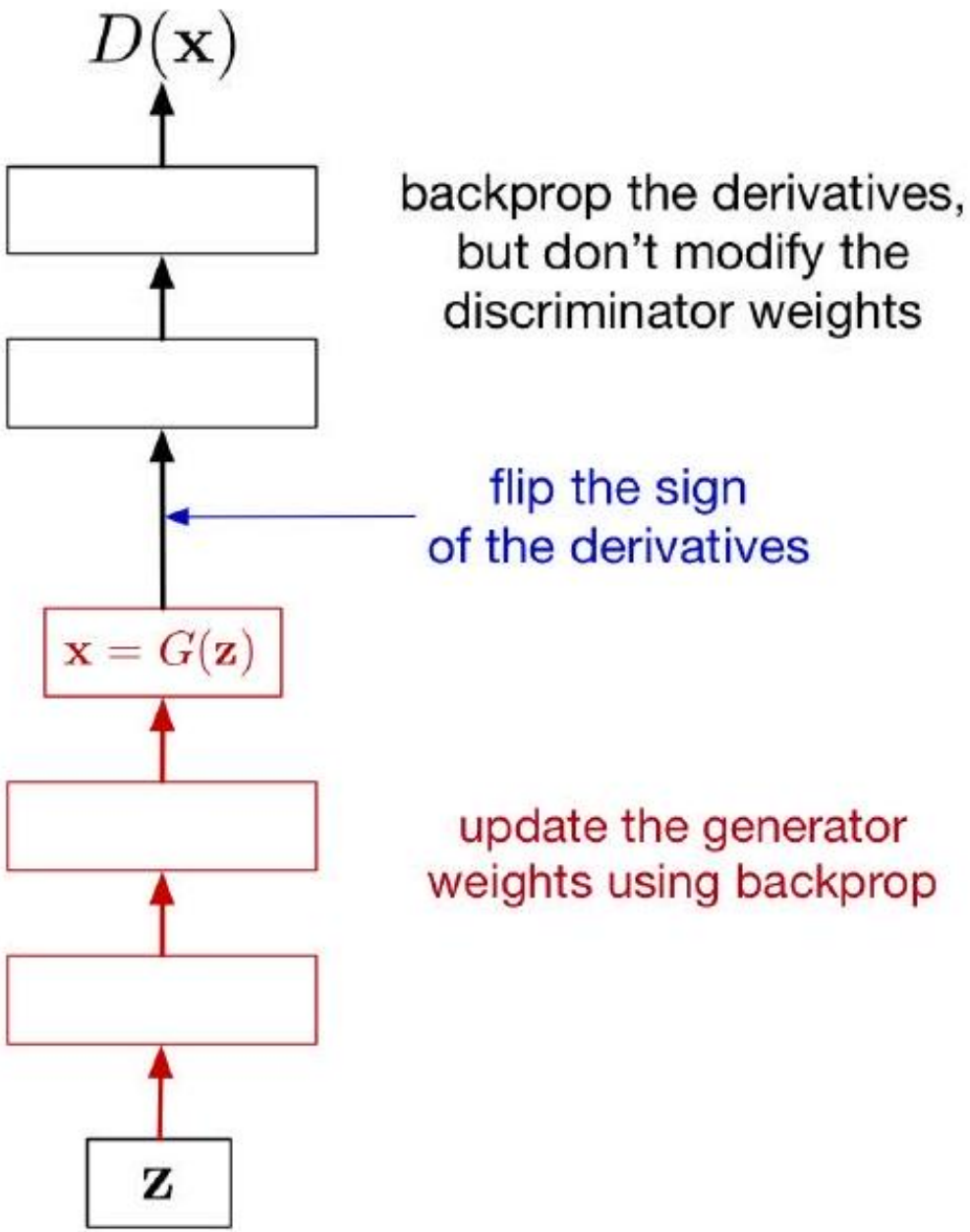
- This is called the minimax formulation, since the generator and discriminator are playing a zero-sum game against each other: $\max_G \min_D \mathcal{J}_D$

# Generative Adversarial Networks

Updating the discriminator:



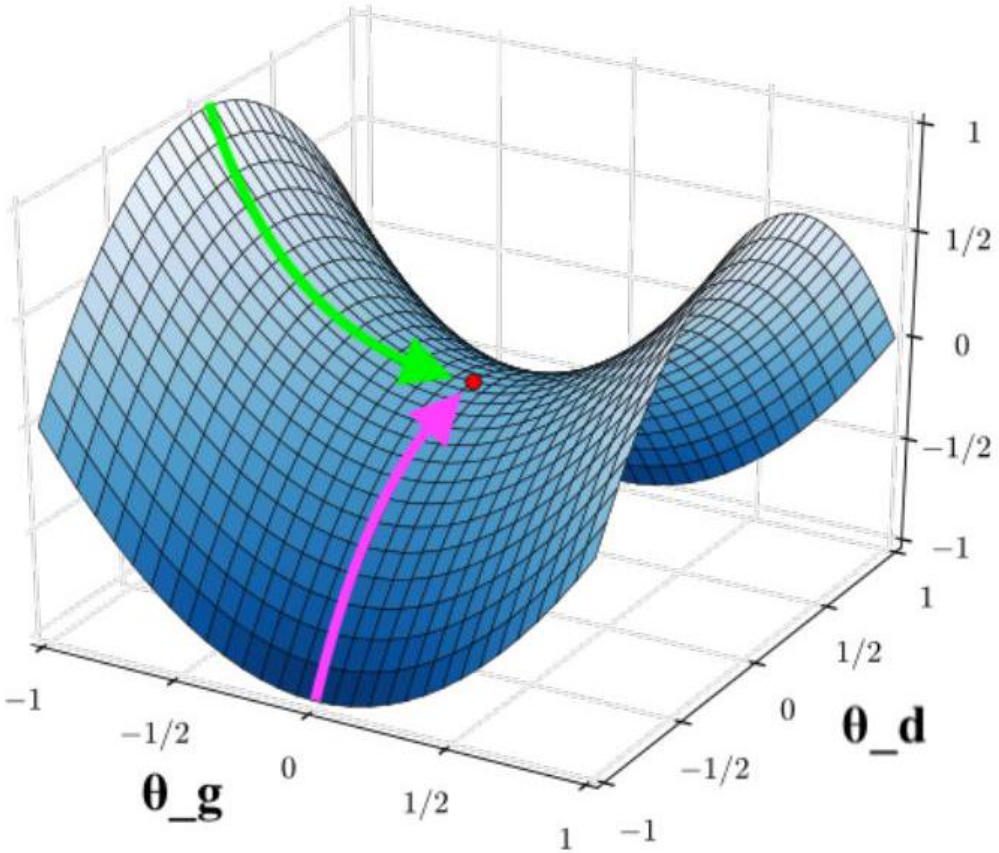
# Generative Adversarial Networks



# GANs: Saddle-Point Optimization

Saddle-Point Optimization: learn $G_{\theta_g}(\mathbf{z})$ and $D_{\theta_d}(\mathbf{x})$ jointly via the objective $\mathcal{J}(\theta_d, \theta_g)$ :

$$\min_{\theta_g} \max_{\theta_d} \underbrace{\mathbb{E}_{p_{\text{data}}}\left[\log D_{\theta_d}(\mathbf{x})\right]}_{\text{likelihood of true data}} + \underbrace{\mathbb{E}_{p_{\mathbf{z}}(\mathbf{z})}\left[\log\left(1 - D_{\theta_d}(G_{\theta_g}(\mathbf{z}))\right)\right]}_{\text{likelihood of generated data}}$$



# GANs: Optimal Discriminators

Claim: Given $G_{\theta_g}$ defining an implicit distribution $p_g = p(\mathbf{x} \mid \theta_g)$, the optimal descriminator is

$$D^*(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_{\text{g}}(\mathbf{x})}$$

**Proof Sketch:**

$$\begin{aligned}\mathcal{J}(\theta_d, \theta_g) &= \int_{\mathcal{D}} p_{\text{data}}(\mathbf{x}) \log D(\mathbf{x}) d\mathbf{x} + \int_{\tilde{\mathcal{D}}} p(\mathbf{z}) \log\left(1 - D\left(G_{\theta_g}(\mathbf{z})\right)\right) d\mathbf{z} \\ &= \int_{\mathcal{D} \cup \tilde{\mathcal{D}}} p_{\text{data}}(\mathbf{x}) \log D(\mathbf{x}) + p_g(\mathbf{x}) \log(1 - D(\mathbf{x})) d\mathbf{x}\end{aligned}$$

Maximizing the integrand for all x is sufficient and gives the result

# Optimal Discriminators Cont.

The integrand

$$h(D(\mathbf{x})) = p_{\text{data}}(\mathbf{x}) \log D(\mathbf{x}) + p_g(\mathbf{x}) \log(1 - D(\mathbf{x}))$$

is concave for $D(\mathbf{x}) \in (0, 1)$. We take the derivative and compute a stationary point in the domain:

$$\frac{\partial h(D(\mathbf{x}))}{\partial D(\mathbf{x})} = \frac{p_{\text{data}}(\mathbf{x})}{D(\mathbf{x})} - \frac{p_g(\mathbf{x})}{1 - D(\mathbf{x})} = 0$$

$$\Rightarrow D(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_{\text{g}}(\mathbf{x})}$$

This minimizes the integrand over the domain of the discriminator, completing the proof.

The optimal discriminator estimates the probability that a sample comes from the real data distribution.

# GANs: Jensen-Shannon Divergence and Optimal Generators

Given an optimal discriminator $D^*(\mathbf{x})$, the generator objective is

$$
\begin{aligned}
C(\theta_g) &= \mathbb{E}_{p_{\text{data}}}\left[\log D^*_{\theta_d}(\mathbf{x})\right] + \mathbb{E}_{p_g(\mathbf{x})}\left[\log\left(1 - D^*_{\theta_d}(\mathbf{x})\right)\right] \\
&= \mathbb{E}_{p_{\text{data}}}\left[\log \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_{\text{g}}(\mathbf{x})}\right] + \mathbb{E}_{p_g(\mathbf{x})}\left[\log \frac{p_g(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_{\text{g}}(\mathbf{x})}\right] \\
&\propto \underbrace{\frac{1}{2} KL\left(p_{\text{data}} \,\|\, \frac{(p_{\text{data}} + p_g)}{2}\right) + \frac{1}{2} KL\left(p_g \| \frac{(p_{\text{data}} + p_g)}{2}\right)}_{\text{Jensen-Shannon Divergence}}
\end{aligned}
$$

$C(\theta_g)$ achieves its global minimum at $p_g = p_{\text{data}}$ given an optimal discriminator!

# Training Procedure

**Algorithm 1** Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, $k$, is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

**for** number of training iterations **do**

**Discriminator updates**

**for** $k$ steps **do**

- Sample minibatch of $m$ noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
- Sample minibatch of $m$ examples $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\boldsymbol{x})$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} \left[ \log D\left(\boldsymbol{x}^{(i)}\right) + \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right) \right].$$

**end for**

**Generator updates**

- Sample minibatch of $m$ noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right).$$

**end for**

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

# Problems with GANs

- Vanishing gradients: the discriminator becomes "too good" and the generator gradient vanishes.
- Non-Convergence: the generator and discriminator oscillate without reaching an equilibrium.
- Mode Collapse: the generator distribution collapses to a small set of examples.
- Mode Dropping: the generator distribution doesn't fully cover the data distribution.

# A Better Cost Function

- We introduced the minimax cost function for the generator:

$$\mathcal{J}_G = \mathbb{E}_{\mathbf{z}}[\log(1 - D(G(\mathbf{z})))]$$

- One problem with this is saturation.
- Here, if the generated sample is really bad, the discriminator's prediction is close to 0 , and the generator's cost is flat.

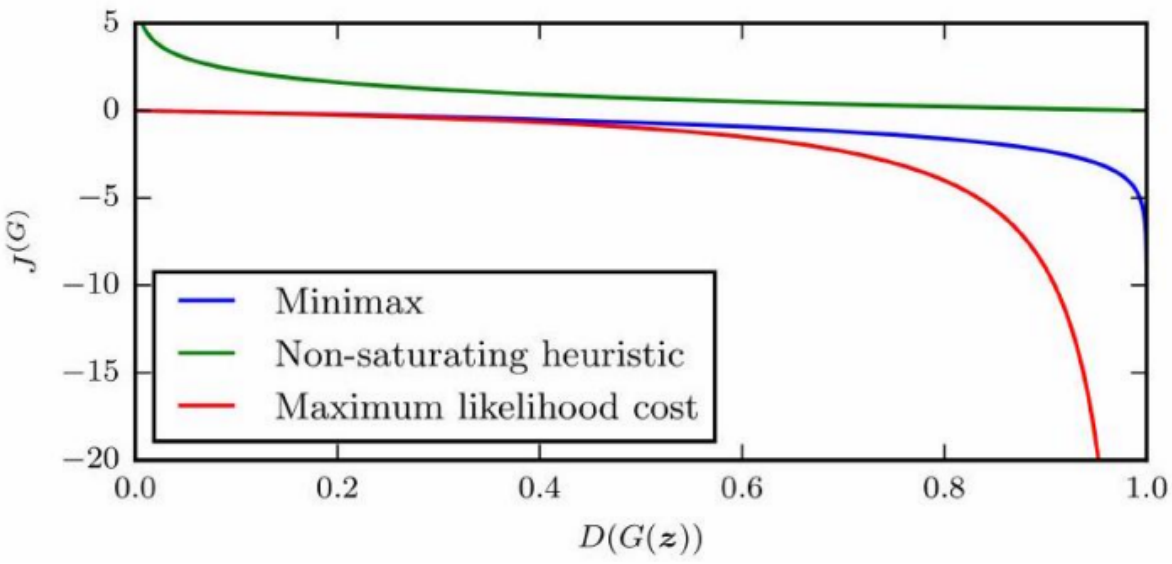
# A Better Cost Function

- Original minimax cost:

$$\mathcal{J}_G = \mathbb{E}_{\mathbf{z}}[\log(1 - D(G(\mathbf{z})))]$$

- Modified non-saturating heuristic generator cost:
- stronger gradients for the generator when the discriminator easily rejects generated samples.

$$\mathcal{J}_G = \mathbb{E}_{\mathbf{z}}[-\log D(G(\mathbf{z}))]$$



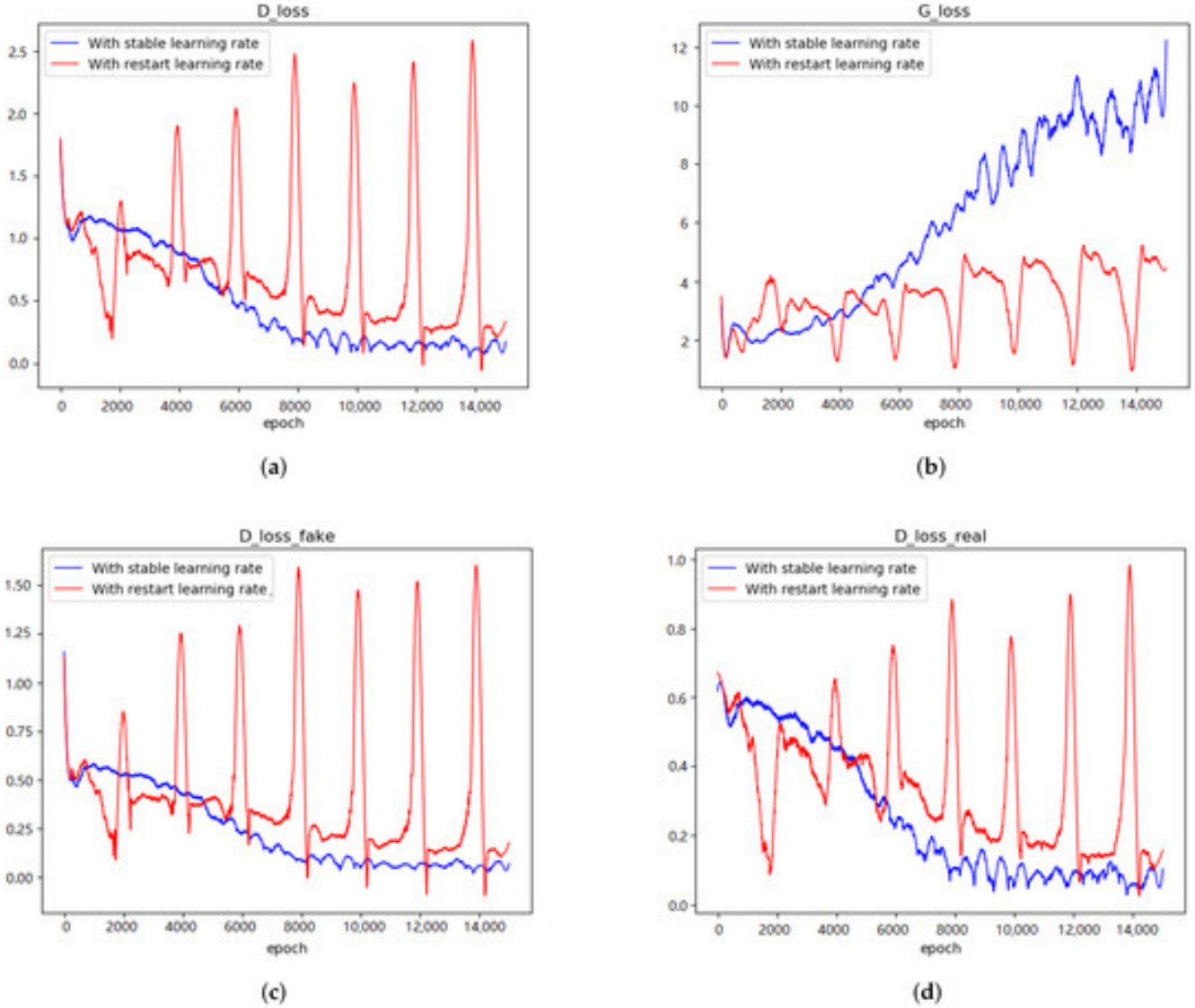
# Problems: Addressing Vanishing Gradients

Solutions:

- Change Objectives: use the non-saturating heuristic objective, maximum-likelihood cost, etc.
- Limit Discriminator: restrict the capacity of the discriminator.
- Schedule Learning: try to balance training $D_{\theta_d}$ and $G_{\theta_g}$.

# Problems: Non-Convergence

Simultaneous gradient descent is not guaranteed to converge for minimax objectives.



# Problems: Non-Convergence

- Goodfellow et al. only showed convergence when updates are made in the function space.
- The parameterization of $D_{\theta_d}$ and $G_{\theta_g}$ results in highly non-convex objective.
- In practice, training tends to oscillate.

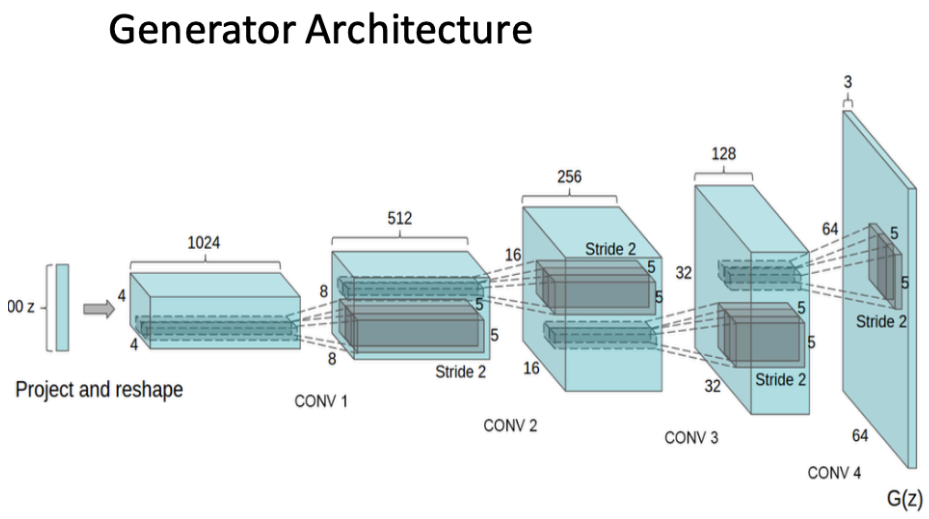
**Addressing tricks:**

- Use DCGAN
- Use stability tricks from RL
- Use SGD for discriminator and ADAM for generator

# DC-GAN

- Most modern architectures are based on the Deep Convolutional GAN (DC-GAN).
- GAN Zoo: https://github.com/hindupuravinash/the-gan-zoo

## Deep Convolutional GANs (DCGANs)




**Key ideas**:

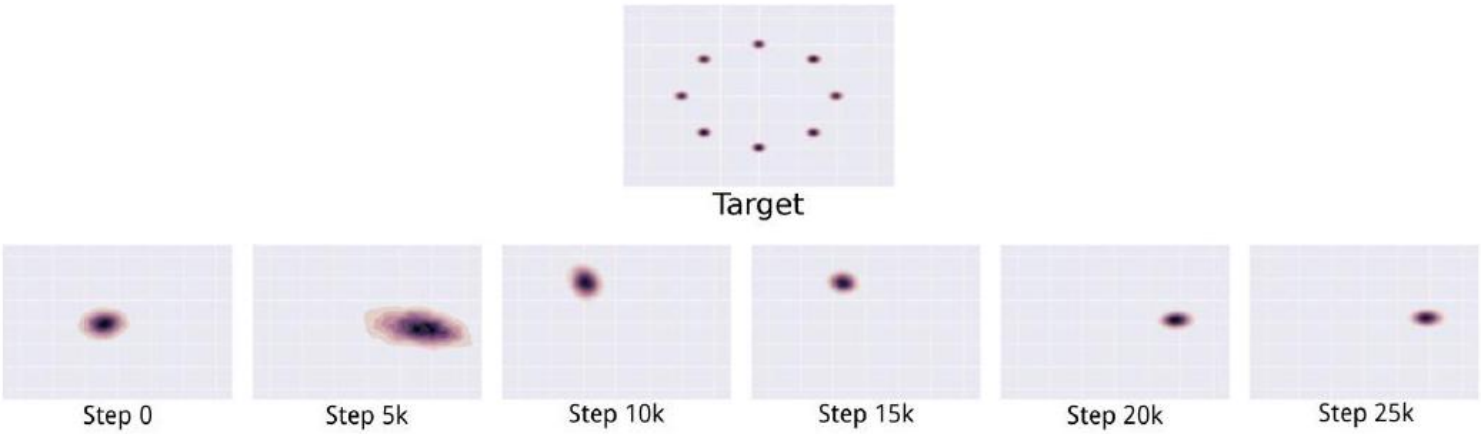
- Replace FC hidden layers with Convolutions
  - **Generator:** Fractional-Strided convolutions
- Use Batch Normalization after each layer
- **Inside Generator**
  - Use ReLU for hidden layers
  - Use Tanh for the output layer

# Problems: Mode Collapse and Mode Dropping

One Explanation: SGD may optimize the max-min objective

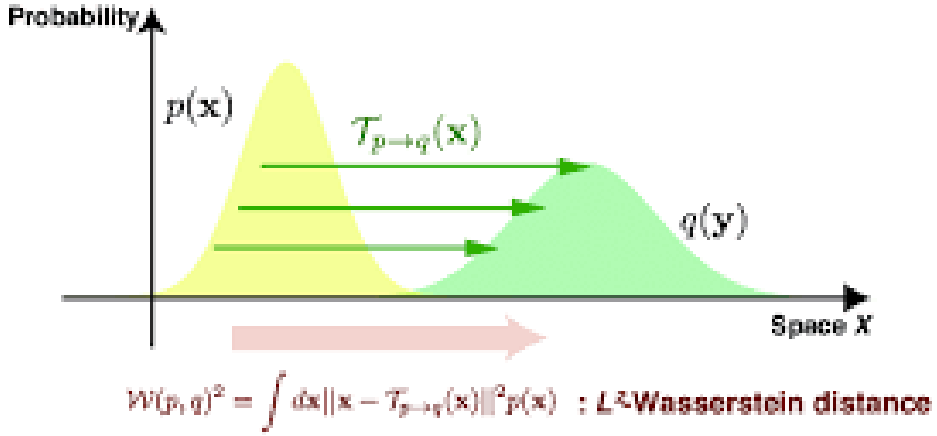
$$\max_{\theta_d} \min_{\theta_g} \mathbb{E}_{p_{\text{data}}} \left[\log D_{\theta_d}(\mathbf{x})\right] + \mathbb{E}_{p_{\mathbf{z}}(\mathbf{z})} \left[\log \left(1 - D_{\theta_d}\left(G_{\theta_g}(\mathbf{z})\right)\right)\right]$$

Intuition: the generator maps all $\mathbf{z}$ values to the $\hat{\mathbf{x}}$ that is mostly likely to fool the discriminator.



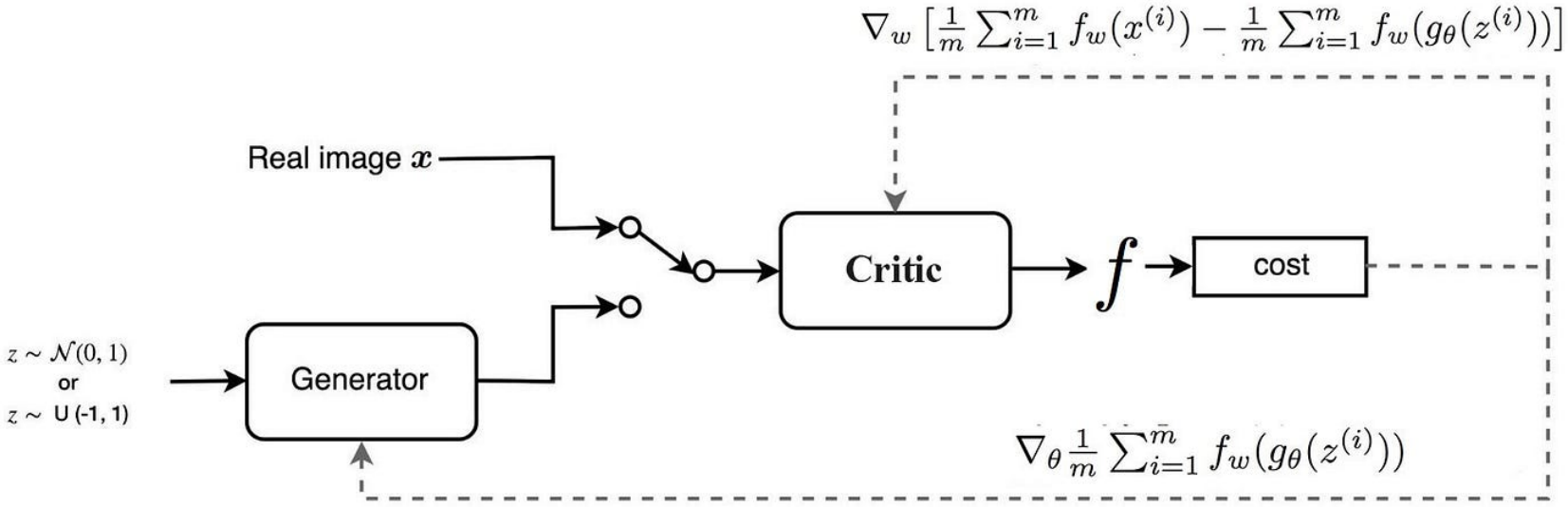
# Wasserstein GANs

The Wasserstein distance (or Earth Mover's distance) intuitively represents the minimum "cost" required to transform one distribution into another.




WGAN

# WGANs


$\nabla_w \left[\frac{1}{m}\sum_{i=1}^{m} f_w(x^{(i)}) - \frac{1}{m}\sum_{i=1}^{m} f_w(g_\theta(z^{(i)}))\right]$
Real image $x$
Critic
$f$
cost
$z \sim \mathcal{N}(0,1)$
or
$z \sim$ U (-1, 1)
Generator
$\nabla_\theta \frac{1}{m}\sum_{i=1}^{m} f_w(g_\theta(z^{(i)}))$

Reinforcement Learning Recap

Introduction to Generative AI
- Autoencoders
- Generative Adversarial Networks (GAN)

Applications in Finance

Challenges of Generative AI in Finance

# Applications I: Data & Market Simulation

**Goal:** Overcome data scarcity and build high-fidelity environments.

- **Synthetic Data Generation & Augmentation:**
  - Generate realistic financial time series and order book data.
  - Enhance limited or imbalanced datasets to improve model robustness.
- **Market Microstructure Simulation:**
  - Create controlled, interactive environments capturing non-linear market dynamics.
  - *Crucial for RL:* Simulates realistic order book dynamics for training algorithmic trading agents. $\Rightarrow$ e.g., MARS (MSFT Asia).

# Applications II: Advanced Financial Tasks

**Goal:** Enhance risk assessment and complex pricing mechanisms.

- **Scenario Generation for Stress Testing:**
  - Generate diverse, extreme, yet mathematically plausible market scenarios (tail risks).
  - Allows portfolio managers to test risk management strategies under conditions not seen in historical data.
- **Volatility Surface Modeling:**
  - Capture complex relationships that traditional stochastic models (like Black-Scholes) may miss.
  - Model intricate pricing surfaces and volatility smiles.
- **Portfolio Optimization:**
  - Understand how assets co-move during generated extreme market regimes, leading to more resilient asset allocation.

Reinforcement Learning Recap

Introduction to Generative AI
- Autoencoders
- Generative Adversarial Networks (GAN)

Applications in Finance

Challenges of Generative AI in Finance

# Overview

Group discussion

# Overview

Group discussion

- Evaluation in generative AI for finance is multifaceted.

# Challenge 1: The Evaluation Bottleneck

- **Absence of Benchmarks:**
  - Unlike Computer Vision (which relies on FID or Inception Score), finance lacks standardized metrics for synthetic data quality.
- **Multidimensional Quality Criteria:**
  - Synthetic data must reproduce key stylized facts: heavy tails, volatility clustering, and cross-asset dependence.
- **The Diversity vs. Fidelity Trade-off:**
  - *Fidelity:* Samples must reproduce key statistical properties of market microstructure.
  - *Diversity:* Must avoid "mode collapse" to generate novel scenarios, not just memorize historical trajectories.

# Challenge 2: Alignment with Financial Objectives

- **Task-Specific Utility:**
  - High statistical similarity does not guarantee economic usefulness. Evaluations must assess impact on downstream tasks like RL-based trading or portfolio optimization.
- **Stress Testing & Tail Risks:**
  - Models must generate plausible extreme market events (black swans) for robust risk assessment.
- **No-arbitrage constraints**
  - Generated data may violate financial constraints such as no-arbitrage conditions
- **Regulatory & Compliance constraints:**
  - avoid generating patterns that correspond to manipulative trading behaviors

# Challenge 3: Market Dynamics and Computational Scale

- **Non-Stationary Regimes:**
  - Market evolution renders historical data obsolete. Generative models must adapt to distribution shifts and unseen market regimes (e.g., pre- vs. post-COVID dynamics).
- **High-Dimensionality:**
  - Modeling multi-asset universes (e.g., cross-asset graphs) or high-frequency Limit Order Book (LOB) data imposes massive computational burdens.
- **Simulation-Based Training Costs:**
  - Running interactive, step-by-step market simulators to train algorithmic agents requires significant low-latency computational resources.